

06. DCGAN & GAN Training

Overview

DCGAN architectural rules (strided convolutions, batch norm, no pooling) make convolutional GANs trainable on image data. We cover discriminator/generator symmetry, learning rate ratios, and monitoring tricks when samples look noisy or collapsed. Training dynamics on MNIST at 4GB VRAM are spelled out.

Lab: Lab 06.

Prior modules: Module 05.

Contents

1	Convolutional GANs	2
2	DCGAN Architecture Rules	2
3	Generator Structure	2
4	Discriminator as Feature Extractor	2
5	Training Diagnostics	2
6	Advanced Stabilizers	3
7	Lab	3
8	Transposed Convolution Arithmetic	3
8.1	Minibatch Discrimination	3
9	Extended Intuition	3
9.1	Receptive Field and Capacity Balance	4
10	Numerical Walkthrough	4
11	Local GPU Constraints	4
12	Debugging Checklist	4
13	Local GPU Constraints	5
14	Debugging Checklist	5
15	Practice Problems	5

Module track

This module builds on **Module 05**. Read those PDFs first. References to other modules appear as **Module NN** (not page numbers, because each module is its own PDF).



Figure 1: Module sequence (solid box: this PDF; dashed: typical next step).

1 Convolutional GANs

DCGAN [4] applies strided convolutions to scale GANs [2] to high-resolution images. Building on **Module 05** (GAN architecture), both G and D become deep CNNs.

2 DCGAN Architecture Rules

1. Replace pooling with strided convolutions (D) and transposed convolutions (G).
2. Use BatchNorm in both G and D (except G output and D input).
3. Use ReLU in G (except output tanh); use LeakyReLU(0.2) in D .
4. Use tanh output scaled to image range; no fully connected hidden layers.

Stable feature matching. Guidelines reduce mode collapse described in **Module 05** (mode collapse) and help D provide informative gradients.

3 Generator Structure

G upsamples latent $\mathbf{z} \in \mathbb{R}^{100}$ through layers:

$$\mathbf{h}_0 = \mathbf{W}_0 \mathbf{z}, \quad \mathbf{h}_{\ell+1} = \text{BN}(\text{ConvTranspose}(\mathbf{h}_\ell)). \quad (1)$$

Typical progression: $4 \times 4 \rightarrow 8 \times 8 \rightarrow \dots \rightarrow 64 \times 64$.

4 Discriminator as Feature Extractor

D outputs scalar logit via conv stack ending in global average pooling:

$$D(\mathbf{x}) = \sigma\left(\mathbf{w}^\top \text{GAP}(\text{Conv}(\mathbf{x}))\right). \quad (2)$$

Table 1: DCGAN layer template (64×64 RGB).

Layer	Kernel	Stride	Channels
G : ConvTranspose	4×4	2	$512 \rightarrow 256 \rightarrow 128 \rightarrow 3$
D : Conv	4×4	2	$3 \rightarrow 128 \rightarrow 256 \rightarrow 512$

5 Training Diagnostics

Track inception score, FID, and visual grids of $G(\mathbf{z})$ for fixed $\mathbf{z}$. If D loss $\rightarrow 0$ instantly, reduce D capacity or add label smoothing.

$$\mathcal{L}_D = -\mathbb{E}[\log D(\mathbf{x})] - \mathbb{E}[\log(1 - D(G(\mathbf{z})))] \quad (3)$$

6 Advanced Stabilizers

Spectral normalization bounds Lipschitz constant of D ; gradient penalty (WGAN-GP) penalizes $\|\nabla_{\hat{\mathbf{x}}} D(\hat{\mathbf{x}})\|_2 \approx 1$.

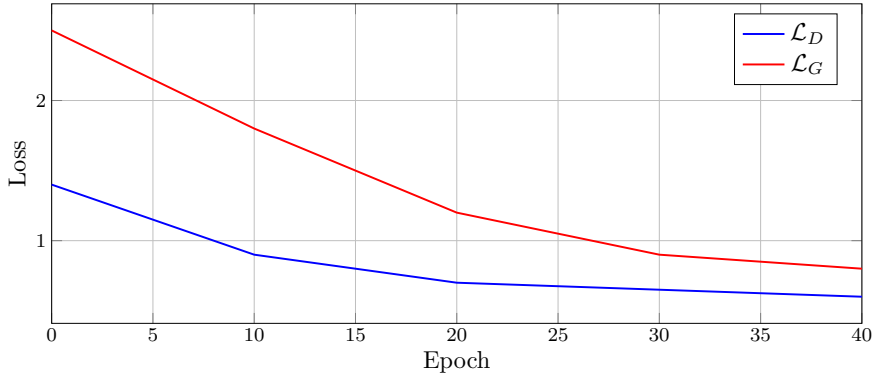


Figure 2: Typical GAN training curves: D and G losses need not converge to zero.

7 Lab

Lab 06 implements DCGAN on a face dataset; verify section 2 and compare sample quality to **Lab 05**. Diffusion in **Module 07** (forward diffusion) trades adversarial training for denoising objectives.

8 Transposed Convolution Arithmetic

Output size for transposed conv: $o = (i - 1)s - 2p + k + p_{\text{out}}$. Stack depth grows channel count while halving spatial dims in D , mirroring G in reverse.

8.1 Minibatch Discrimination

Add layer to D encouraging diverse samples within a batch, reduces mode collapse from **Module 05** (mode collapse).

9 Extended Intuition

DCGAN replaces fully connected generators with strided transposed convolutions that upsample spatial feature maps. The discriminator uses strided convolutions instead of pooling, so both networks learn their own downsampling/upsampling filters. Batch normalization in G and D (except G output and D input) keeps activations in a range where gradients flow.

Architectural rules matter more than loss tweaks on MNIST/CelebA-scale runs: no pooling, use LeakyReLU ($\alpha = 0.2$) in D , ReLU in G , tanh output. Generator noise $\mathbf{z} \in \mathbb{R}^{100}$ is projected to $4 \times 4 \times 512$ feature maps, then upsampled through 8×8 , 16×16 , to 64×64 . Symmetry between G and D depth helps balance capacity so neither side dominates instantly.

Learning rate ratios show up in practice: some trainers use $\text{lr} = 2 \times 10^{-4}$ for both; others slightly lower D . Monitor mode collapse with per-class diversity if labels exist, or with nearest-neighbor checks in pixel space (crude but fast). Diffusion models in **Module 07** largely replaced GANs for high-fidelity image synthesis, but DCGAN remains the best lab for adversarial dynamics.

9.1 Receptive Field and Capacity Balance

Each strided conv in D doubles the effective receptive field; D should see at least the full image by the deepest layer or it judges only local patches. If G is much deeper than D , D cannot detect long-range inconsistencies (color gradients spanning the image); match channel counts and depth roughly symmetrically. Spectral normalization on D conv weights (optional extension) bounds Lipschitz constant and reduces D overpowering; not in the base lab but common in modern conv GANs. Mini-batch discrimination concatenates pairwise distances within a batch to D input, penalizing G for producing identical samples; useful when collapse persists after architectural fixes. Transposed conv artifacts stack across layers; if checkerboard appears only at high frequency, reduce stride to 1 on the final upsample block as a targeted fix.

10 Numerical Walkthrough

64×64 RGB DCGAN, $\mathbf{z} \in \mathbb{R}^{128}$, $B = 32$, channels (64, 128, 256, 512) in D .

G parameter-heavy block: `ConvTranspose2d(512, 256, 4, 2, 1)` maps $4 \times 4 \times 512 \rightarrow 8 \times 8 \times 256$. Three more upsample stages reach $64 \times 64 \times 3$. Total G params often 5 to 12M depending on width; D similar.

Forward fake: $\mathbf{z}$ shape (32, 128, 1, 1) $\rightarrow$ (32, 3, 64, 64) output in $[-1, 1]$. Real images from dataloader normalized with `transforms.Normalize((0.5,)*3, (0.5,)*3)`.

One training iteration at $\text{lr} = 2 \times 10^{-4}$: update D on real batch, then fake (detached), then G to fool D . VRAM estimate: activations at 64×64 with $B = 32$ land around 2 to 3 GB with Adam; drop to $B = 16$ if OOM.

Spatial size check for G : $4 \rightarrow 8 \rightarrow 16 \rightarrow 32 \rightarrow 64$ requires four stride-2 transpose convs with kernel 4, padding 1. D mirrors with four stride-2 convs: $64 \rightarrow 32 \rightarrow 16 \rightarrow 8 \rightarrow 4$, then flatten to a linear logit. If your G output is 62×62 , one transpose block has wrong padding; fix before tuning lr for 1000 steps.

Frechet Inception Distance (FID) needs 10k+ samples and a pretrained Inception network; for lab-scale runs, pixel-space mean/variance distance is a crude but fast sanity check. Color shift in generated RGB: G bias terms dominate when D is weak; inspect channel means of fake batch vs real batch before blaming mode collapse.

11 Local GPU Constraints

64×64 , $B = 32$ is the practical ceiling on 4 GB; $B = 16$ with accumulation to effective 32 is safer. 128×128 faces need width multiplier 0.5 or progressive growing (not in base lab).

fp32 training recommended; fp16 batch norm stats can jitter on small batches. Pin memory and `non_blocking=True` transfers shave idle time on a 1650.

Progressive growing (ProGAN) trains at 4×4 then doubles resolution; out of scope for the base lab but explains why jumping straight to 128×128 DCGAN fails on limited data. Instance norm in G (StyleGAN-style) removes batch dependence at small B ; batch norm at $B = 16$ can jitter running stats and destabilize D . Track FID proxy (pixel mean L2 between real and fake batches) every epoch; sudden drops in distance without visual improvement means D is gaming your metric.

12 Debugging Checklist

- `ConvTranspose2d(kernel=4, stride=2, padding=1)` doubles spatial size for standard DCGAN blocks.
- `nn.BatchNorm2d` in G ; in D , use `LeakyReLU(0.2, inplace=True)` between conv layers.

- Weight init: `nn.init.normal_(m.weight, 0.0, 0.02)` for conv and linear in both nets (Radford et al.).
- Do not apply batch norm to G final layer; D first layer often skips norm on input conv.
- `torchvision.utils.save_image` expects $[0, 1]$; denormalize from tanh range first.

```
def weights_init(m):
    if isinstance(m, (nn.Conv2d, nn.ConvTranspose2d, nn.BatchNorm2d)):
        nn.init.normal_(m.weight, 0.0, 0.02)
G.apply(weights_init); D.apply(weights_init)
```

13 Local GPU Constraints

64×64 , $B = 32$ is the practical ceiling on 4 GB; $B = 16$ with accumulation to effective 32 is safer. 128×128 faces need width multiplier 0.5 or progressive growing (not in base lab).

fp32 training recommended; fp16 batch norm stats can jitter on small batches. Pin memory and `non_blocking=True` transfers shave idle time on a 1650.

14 Debugging Checklist

- Checkerboard pattern: stride/kernel mismatch; verify transpose conv output size formula $(in - 1)s - 2p + k$.
- Washed-out colors: forgot tanh or wrong normalization on real images.
- D accuracy 100%: G not learning; try label smoothing or extra G steps.
- OOM at 64×64 : halve B or reduce base channel count from 512 to 256 in deepest layer.
- Same face every sample: collapse; increase $\|\mathbf{z}\|$ dim to 256 or add mini-batch discrimination (advanced).
- High-frequency noise in D feature maps: leaky slope too small or missing weight init at 0.02 std.
- G gradients zero on fake batch: forgot to remove `detach()` when backpropagating through D into G .
- Discriminator loss exactly 0.693: outputs stuck at 0.5 everywhere; check for label inversion or frozen G accidentally updating D graph.

15 Practice Problems

Problem 1. Compute spatial sizes through G starting from $4 \times 4 \times 512$ with three stride-2 transpose convs.

Problem 2. Count parameters in first G layer: `ConvTranspose2d(512, 256, 4, 2, 1)` including bias.

Problem 3. Train DCGAN on MNIST resized to 64×64 and compare visual quality to MLP-GAN from **Lab 05**.

Problem 4. Ablate batch norm in G (one run without) and document training stability.

Problem 5. Sample 100 fake images, compute pairwise ℓ_2 distance in pixel space. Compare mean distance before and after collapse.

Problem 6. Ablate weight init (default PyTorch vs 0.02 normal): document first-epoch D accuracy and sample grid at step 2000.

Complete **Lab 06**; forward diffusion in **Module 07** defines a non-adversarial generative objective.

Source certification

This module lists where each core definition, equation, figure, and summary table comes from. Pedagogical simplifications (lab batch sizes, epoch counts, illustrative plots) are labeled in extension sections and are not claimed as optimal production settings.

Table 3: Certified topic map for Module 06.

Topic	Primary source	Where in this PDF
DCGAN architectural guidelines	[4]	Section 2
GAN objective (prerequisite)	[2]	Section 1
Wasserstein GAN context	[1]	Extension section

Full bibliographic entries appear below. Figures are schematic unless a caption cites a specific paper figure. If a statement is not listed here, treat it as general ML practice or lab-specific guidance, not a cited theorem.

References

- [1] Martin Arjovsky, Soumith Chintala, and Léon Bottou. Wasserstein generative adversarial networks. In *ICML*, 2017.
- [2] Ian Goodfellow, Jean Pouget-Abadie, Mehdi Mirza, et al. Generative adversarial nets. In *NeurIPS*, 2014.
- [3] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *ICLR*, 2015.
- [4] Alec Radford, Luke Metz, and Soumith Chintala. Unsupervised representation learning with deep convolutional generative adversarial networks. In *ICLR*, 2016.
- [5] Tim Salimans, Ian Goodfellow, Wojciech Zaremba, et al. Improved techniques for training GANs. In *NeurIPS*, 2016.